

Pádel & Gol

Sistema de Gestión de Reservas Deportivas

Informe de desarrollo — TP3

Morán Santiago / Fakiani Octavio | AW2 — IES 21 — 2026

¿Qué se hizo en este TP?

En el TP3 reemplazamos los datos que veníamos usando desde MockAPI por una base de datos real. Se armó el CRUD completo de canchas (alta, baja, modificación y dos lecturas) y se conectó todo a una base de datos PostgreSQL alojada en Supabase.

¿Por qué Supabase?

Supabase nos da una base de datos PostgreSQL gratuita en la nube, sin tener que instalar nada localmente. Solo hace falta crear el proyecto en Supabase y copiar la cadena de conexión para que el servidor se conecte a la base.

¿Cómo se conecta el servidor a la base?

El servidor (Node.js + Express) se conecta directamente a la base de datos de Supabase usando el paquete pg, con un Pool de conexiones. La cadena de conexión (DATABASE_URL) se guarda en un archivo .env que no se sube al repositorio, así no quedan expuestas las credenciales. Al iniciar el servidor, se verifica la conexión y se crea la tabla canchas si todavía no existe.

Estructura del proyecto (patrón MVC)

- config/db.mjs: configura la conexión a Supabase
- models/Cancha.mjs: tiene las consultas SQL a la tabla canchas
- controllers/canchaController.mjs: recibe la petición y decide qué responder
- routes/: define las rutas y las conecta con el controlador
- front/: HTML, CSS y JS del sitio y del panel de administración

Endpoints de la API

API para administrar (/api/canchas):

- GET /api/canchas: lista todas las canchas
- GET /api/canchas/:id: trae una cancha puntual
- POST /api/canchas: crea una cancha nueva
- PUT /api/canchas/:id: actualiza una cancha
- DELETE /api/canchas/:id: elimina una cancha

API pública de solo lectura (/api/web), para el sitio:

- GET /api/web/canchas: lista las canchas para mostrar en la web
- GET /api/web/canchas/:id: detalle de una cancha

Panel de administración

Se armó una página (admin.html) donde se pueden ver todas las canchas en una tabla, agregar una nueva con un formulario, editar una existente con un modal, y eliminarla con confirmación. Todo esto consume la API CRUD que mencionamos arriba.

Manejo de errores

Se usa try/catch tanto en el modelo como en el controlador. El modelo atrapa los errores de las consultas a la base de datos y los muestra en consola con el nombre de la función donde ocurrieron, y el controlador decide qué mensaje y código de error devolver al cliente (por ejemplo, 404 si la cancha no existe, o 400 si los datos no son válidos).

Corrección que hicimos después de la devolución del profe

Al principio, el front-end de la página de canchas intentaba conectarse directamente a Supabase desde el navegador, y solo usaba nuestra propia API si esa conexión fallaba. Lo corregimos para que el front-end use siempre la API que armamos nosotros, y que sea el servidor (no el navegador) el que hable con Supabase. Así seguimos la arquitectura que se pidió: front-end → API propia → base de datos.

¿Cómo se corre el proyecto?

Hay que crear un archivo .env con la URL de conexión a Supabase (DATABASE_URL) y el puerto. Después, dentro de la carpeta del proyecto:

```
npm install
```

```
npm run dev
```

Y se abre el navegador en <http://localhost:3000>

Qué queda pendiente

Las reservas todavía no se migraron a la base de datos, por ahora solo está armado el CRUD de canchas. La idea es sumar la tabla de reservas y su lógica en una próxima etapa.